上位机（`host/`）S1–S5 全面自查报告

- **自查日期**：2026-10-13（21:30 起）
- **触发**：用户要求「先全面自查一遍」（`m03503`）
- **对象**：`host/` 全部 23 个 Python 文件（S1 链路段一直到 S5 记录/导出与界面级自检）
- **硬件基线**：板上 **v12** 位流、`COM3 @ 2 Mbaud`、输入 `f_in ≈ 19,999.900 Hz`、板上配置 `inter=32 div=1 极对数=4`
- **详细日志**：逐条修复记录与取证见 `docs/DEV_STATUS.md` **§1.66**；本报告是一份可独立阅读的汇总

---

1. 一句话结论

26 处真缺陷已修复（其中 5 处会直接给出错数、丢数据或崩溃）、**101 项离线自检 + 倍频 2043 组往返全部 PASS**、 **真机 S1/S2/S3/S4/S5/拔插/界面级核对全部 PASS**、`link.py` **协议层重写后重跑 30 分钟长稳 15/15 PASS**； 另有 4 项已知但不阻塞验收的遗留 / 口径澄清项已逐条记录（见 §7）。 自查过程中**改掉了 2 条判据的严苛程度、新增 1 条盘空间判据**，两条都如实说明原因（见 §5），没有为了让自检变绿而放宽结论性判据。

---

2. 自查方法与范围

不做「只读一遍代码」式的复查，而是：

1. **六路分片审计**：把 `host/` 的 23 个文件切成 6 片交给 6 个子代理独立审计 （`main_window.py` | `abz.py` + `export_png.py` | `encoder.py` + `signal.py` + `regs.py` | `recorder.py` + `panel_record.py` | `link.py` + `app.py` | `panel_*.py` + `theme.py`）， 每条结论必须给出 **文件:行号 + 触发条件 + 后果。**
2. **逐条复核**：子代理报告只当线索，本人对每条都在本机复现或用真机波形/寄存器回读确认， **不成立的结论直接丢弃**（实际丢弃了若干条「看起来像 bug 但 RTL 语义就是那样」的判断）。
3. **修复后取证**：离线自检补用例、真机重跑、并把证据写进 `docs/DEV_STATUS.md` §1.66。

覆盖的 8 个维度：数据口径正确性、崩溃边界、资源上限与清理、线程/队列时序、寄存器读-改-写安全、 文件写出语义、界面状态机、验收判据本身是否可信。

---

3. 修掉的缺陷（按严重度）

3.1 会直接「读错数 / 丢数据 / 崩溃」的 5 处

#	位置	触发	后果	修法	证据
1	`host/encoder.py:204-205`	任何工况（原式 `loss_pct = |mean_pos| /|int_part| - 1`）	只是**取整残差**：零丢帧的 12345 Hz 报 **22.52%**、14000 Hz 报 **40%**（面板还染告警色）；真丢 35% 反而只报 **7.69%**	改用链路时隙真值 `1 - dn/(clock-k0)`（新增 `slots` / `resid_pct` 键），口径切换加**迟滞** 1.5%/3% + `clock_ok` 先验，`span` 随口径取 `dt_clock` / `dt_wall`	自检 [9] 零丢帧不报丢帧（10000/12345/12999/14000/19999.9 Hz 全部 `loss<0.5%`）、[10] 丢帧率口径 = **50.0%**
2	`host/link.py`（`_execute_seq` + `_loop`）	每次写寄存器	判据 `want = max(1, len(payloads)-1)` 在「板子对每条命令立即回一个应答」时永远凑不齐 ⇒ **每次写固定睡满 0.502 s 超时**；`self._cmd_req = None` 还会**静默抹掉**执行期间提交的新命令	改为**每发一帧收一轮**（`_collect()` 带 6 ms 安静窗口）、应答进 `_replies` 队列、`_take_cmd()` / `_cmd_done(req)` 只清自己那条、`transact()` 等待移到 `_cmd_lock` **外**	真机计时：读寄存器 **0.001 s**、写 **0.002 s**（`0x010C` 0.052 s），**原来固定 0.502 s**；`reply_ok=24 reply_timeout=0`
3	`host/regs.py:256-259`（同类 `:239-240` / `:247-248`）	读现值失败（掉线/超时）后写高级参数	`cur or 0` **整字节覆盖**：写 `zwidth` 会清掉板上 `0x0108` 的 bit14（实测板上 `0x0108 = 0x4004`）、`zphase` / `uvwph` 在 `0x010C` 里互相清位	`adv_field_writes()` 读不到现值就**返回 `None`**；`host/panel_interp.py` 逐字段检查，任一缺值就 `adv_blocked` **整批不发**并给中文提示（`host/main_window.py` 接日志）	自检 3 条（`zwidth` / `zphase` 缺值→`None`；现值为 `0` 是正常值≠缺值）
4	`host/main_window.py:582`（`disconnect_port`）	关窗/断开瞬间	原来「先排空、再 `stop()`」，在两者之间串口线程还会推 1~2 批（≈1000 帧），**这几批永远进不了 CSV**	改为 **`w.stop()`（内部 join）→ 再 `_drain_events()`**	真机核对：关窗时队列 2 条 → 关窗后 **0 条**，`记录器行数 - 界面取帧数` **差 0**
5	`host/signal.py`（`evaluate`）	门限读到了、但该轮 `r` 还没算出来（缓冲区刚清空/只读了状态）	`"r=%.0f 正常" % None` 抛 `TypeError: must be real number, not NoneType` —— 在 Qt 槽里就是**整条刷新链断掉**	加 `_r_ok` 判定（`r` 与两端门限都非 `None` 才带数值），否则退回「置位/正常」	新增 2 条自检（**20 → 22 项**）；修复前实测该路径必崩

3.2 统计失真、边界崩溃与语义自相矛盾（16 处）

#	位置	问题	修法
6	`host/link.py` `_pending_reply`	应答只留**最后一条**（同时到多条就丢）	改 `self._replies` 队列

#	位置	问题	修法
7	`host/link.py _read_available`	`read_size=4096` 是**死参数**（只用 `in_waiting`）	作为**单次读取上限**（积压多就读几轮：18 B/帧、50 ms≈9 kB ⇒ 4096 读 3 轮）
8	`host/link.py _emit`	队列满挤掉最老事件时，若那是 batch **不计数** ⇒ 丢帧估计漏报	补记 `dropped_batches` / `dropped_frames`
9	`host/abz.py:69/74/85-88`	`z_width_deg` / `z_duty` 按 `÷div` 折算，而同文件 `synth_frames` 按原始 idx 算 ⇒ **同一快照两个说法**	按 `rtl/abz_gen.v:704`（每个 `idx_exec` 递减 `r_z_left`）+ `rtl/abz_gen.v:41-42/358/422` 改 ∝ **div**；`ab_period_deg(inter, div=1)` / `edge_step_deg(inter, div=1)` 加 `div`；与 `docs/01:596` 的分歧**标注待 ILA 定案**
10	`host/abz.py:261-272`	`_rising()` 把「上一箱无样本」当低电平 ⇒ 没有 AB 跳变也报上升沿（单帧 A=B=0 报相位差 0.00°）	`None` 不算跳变
11	`host/abz.py:109`	`bins=0` ⇒ `snapshot` 除零、`feed` 索引越界	新增 `clean_bins()`（0 → 64）
12	`host/abz.py`（相位锁死）	`f_in` 为 `FRAME_HZ/64 = 156.25 Hz` 整数倍时箱覆盖相位锁死，实测/理论差 **20–34 倍**（20000→25.60×、15000→34.13×、10000→25.60×）	新增 `frame_phase_locked(f_in)` + `duty_z_locked` 快照键 + notes 锁相提示（给出时间口径），自检含「箱数不足时给提示」
13	`host/export_png.py:464-470`	自检 [5] 的 `r5` 在 `try` 内赋值、`try` 外引用 ⇒ 真抛异常时整个自检以 `UnboundLocalError` 崩	先给初值
14	`host/export_png.py:97-100`	`_save` 用**调用方 path** 判成功：无后缀时 matplotlib 写成 `path+".png"` ⇒ 报 `ok=False`/WinError 2；目标是目录时写到同级 `<目录>.png` 却报 `ok=True`	先判 `isdir` → 中文报错；无后缀自动补 `.png`；**用最终路径**算 bytes/px
15	`host/export_png.py:99-100` vs `:327-334`	`int()` 与 `round()` 不一致（dpi=133 差 1 像素）⇒ `selftest_s5` 假失败	统一 `int()`
16	`host/export_png.py:55-65`	`_meta_lines` 的 `if v` 丢 `0`/`False`；键类型混排 `sorted()` 崩；非 dict 崩	`v is not None` + `sorted(key=str)` + 非 dict 兜底
17	`host/recorder.py` 自检	**★用例互相踩名**：[10] 用 `e.csv`、[11] 用 `f.csv` 与 [7]/[8] 撞名，而 `start()` 会用 `_unique_path()` 改名 ⇒「读回构造出来的路径」读到**上一个用例**的文件，报出「跨度 0.000 s」的**假失败**	用例改独立名并读 `rec.files[0]`；另补 [11] 同名不覆盖、[12] 写盘失败自收场（`OSError(28)` 注入）、[13] 上限/时钟钳位 ⇒ **13 项**
18	`host/recorder.py` 自检临时目录	上一轮句柄没释放时 `rmtree(ignore_errors=True)` 静默失败，**残留旧文件被下一轮读到**造成假失败	去掉 `ignore_errors`，删不掉就顺延 `_2/_3…`

#	位置	问题	修法
19	`host/recorder.py` `_open_part`	part>1 时不去重 ⇒ 同秒内分卷可能覆盖	`_unique_path` 对 part>1 也生效（`host/recorder.py:339`）
20	`host/app.py`（5 处）	`--seconds -1` 直接崩；`--export` 缺 `--port`；`--record-dir` 失败静默；`--smoke --seconds 1 --export` 在导出前就退出；截图/连接失败仍 `exit 0`	逐项中文报错 + `exit 2`；`_FAIL` 汇总 ⇒ `exit 1`；`wait_ms ≥ 3500`（有 `--export` 时）
21	`host/panel_alarm.py`	告警 chip `setFixedWidth(108)` 会把「0x053A b0 置位」（实测 164 px）**两端裁掉**	`setMinimumWidth(108)` + `setMaximumWidth(230)`

3.3 状态机与显示残留（5 处）

#	位置	问题	修法
22	`host/panel_record.py` `finish_export`	`info["bytes"]` / `info["px"]` 缺键 ⇒ **KeyError 打进 Qt 槽**	`info.get(...) or 默认值`
23	`host/panel_record.py`	`_refresh_buttons(self._conn, True)` 把按钮**硬顶成可用** ⇒ 断开后仍能在没串口时开表	去掉那个 `True`；`set_dir` 兼收 `ValueError`（含 NUL 的路径）
24	`host/panel_speed.py`	`ok=False` 时只改 note，`v_f/v_rpm/v_turns/v_cycles/v_dir` **挂着上一次的转速**	一并清成「—」
25	`host/ports.py`	任何含 `USB-SERIAL` 的设备都被当成目标板 ⇒ **可能把真正的 CH340 顶掉**	新增 `board_rank()`（VID:PID `1A86:7523` /CH340/CH341 = 0 → `USB-SERIAL` = 1 → 其它 = 2），`preferred_port` 取 rank 最小
26	`host/main_window.py` `_reset_live`	重连后仍留着**上一连接的寄存器快照** ⇒ 拿旧快照做读-改-写	`_reset_live()` 里 `interp.reset()`；`adv_blocked` 接中文日志提示

---

4. 真机复跑记录（COM3 @ 2 Mbaud，板上 v12）

套件	命令	结果
S1 链路	`python host\selftest_link.py --port COM3 --seconds 5`	**PASS exit=0**（读身份证 / 收帧 / 写回读一致：`0x0101 0x0020`、`0x0102 0x0000`）
S2 频率与倍频	`python host\selftest_s2.py --port COM3 --seconds 6`	**PASS**（频率交叉验证 + 倍频写回读）

套件	命令	结果
S3 波形与告警	`python host\selftest_s3.py --port COM3 --seconds 5 --clear`	**PASS**（清锁存实测：写后回读 `0x0404 → 0x00`）
S4 等效采样	`python host\selftest_s4.py --port COM3 --seconds 8`	**12/12 PASS**（自动延长到 16.05 s：AB 箱 **214/256 = 83.6%**、UVW **256/256**、A 占空比 50.5%、A→B 相位 **−90.0°**、非法 UVW **0 帧**、六状态最大偏差 **1.02%**、Z 占空比 0.0552%、`z_window` 置位 2489 帧、方向位 159402/159402、限幅器未激活）
S5 记录+导出	`python host\selftest_s5.py --port COM3 --seconds 12`	**PASS 9/9**（自查后新增「磁盘空间够本次记录」预检，实测 `剩 26.43 GB / 需 0.02 GB`）
S5 拔插	`python host\selftest_s5.py --port COM3 --seconds 12 --reconnect`	**PASS 2/2**（第一次连接收帧 19,944 帧 → 关端口空窗 1 s → 重开同一端口再收 19,957 帧；详见 `docs/DEV_STATUS.md` §1.66）
S5 长稳（协议层重写后）	`python host\selftest_s5.py --port COM3 --soak-minutes 30 --dir host\_s5_soak`	**PASS 15/15**（1800.0 s / **17,859,119 帧 / 2.77 GB / 18 卷 / 9,922 帧每秒 / 丢帧 0.76%**；帧率中位 9841、最低/中位 0.96；**内存 125→86 MB 不涨**；丢批 0、丢帧 0；与重写前基线 17,858,785 帧/2.74 GB/18 卷逐项一致）
界面级（离线）	`python host\selftest_ui.py`（offscreen）	**30/30 PASS exit=0**（含 30 万帧内存增长 +0.0 MB、导出改名、环形缓冲上限、回调异常后按钮解锁）
界面级（真机）	`python host\selftest_ui.py --real-port COM3 --seconds 6`	**PASS exit=0**（三判据全 0 差：`记录器行数 = 界面取帧数`、`文件行数 = 记录器 rows`、`关窗后事件队列 0 条`）
离线自检合计	`encoder 10 + abz 15 + export_png 11 + recorder 13 + signal 22 + ui 30`	**101 项全 PASS**，另 `regs.py` **2043 组**往返 + 5 条边界用例 PASS
语法/编译	`python -m compileall -q host`	**exit=0**

5. 判据变更与诚实说明

自查不只修实现，也发现**两条验收判据本身过严/口径不对**，已改并说明理由：

1. **S4 的「AB 等效采样箱覆盖率」不能按固定时长判**：填箱速度由 `f_in` 与帧率 `FRAME_HZ` 的可通约性决定（板上实测 `f_in ≈ 1006 Hz` + 10000 Hz 帧率时，2 s 只填 **34/256** 箱）。 改为 `--max-seconds`（默认 `max(4×--seconds, 40)` 秒）**填不够自动延长并复用已收帧**， 判据仍是「覆盖足够」而不是「够快」。
2. **S4 的「Z 逐帧命中数」从失败判据降为提示**：Z 常态只有几个 `idx_exec` 拍（≈1 µs）， 而帧周期 100 µs；RTL 的 `Z_TIMEOUT = 4096 clk`（≈82 µs）超时兜底还会改变脉宽 ⇒ 同一工况实测命中 **0 / 88 / 107**

/ 197 / 369 帧都出现过（§1.64 记过 0/16/121/123）。 Z 的「存在性」由「`z_window` 标志出现过」+「Z 脉冲稀疏」+ 离线 `abz.py` 15 项负责。

3. **`abz.py` 的 Z 脉宽口径（`∝ div`）与 `docs/01:596` 冲突**：按 RTL 反向推导取 `∝ div`， 但**没有真机 ILA 直接证据**，已在代码 docstring 与本报告标注「待 ILA 定案」，没有偷偷改文档口径。
4. **`LATCH_CLEAR_VALUE` 决定不收窄**：审计建议收窄到实际会被锁存的位（`0x2D`，因 `rtl/reg_file.v:248-249` 把 b4 恒接 0），但 `rtl/reg_file.v:412-413` 是整字节 W1C， 对永不置位的位写 1 无副作用；反之若将来 RTL 接出 b4，窄掩码就清不掉它 ⇒ 保持 `0x00FF`， 理由写进 `host/signal.py` 注释。
5. **`selftest_link.py` 打印的「参照工具 212 帧校验错」**：那是 2026-10-13 参照测量的数字， 已明确标注为历史参照，本次运行只看「本窗口 N 帧校验错」。
6. **S5 是「加严」而不是放宽**：长稳第一次跑到 1320 s（13,101,391 帧 / 2.03 GB）时我误判磁盘满而中断（沙箱里 `Get-PSDrive C` 报 free 0.0 GB，实际还有 29 GB），于是给 `selftest_s5.py` 新增 **开记录前的盘空间预检**（按 10 kHz ≈ 1.5 MB/s、取 2 MB/s 保守估并留 20% 余量，不够就中文提示 + `exit 1`）⇒ **S5 判据 8 项 → 9 项**，完整 30 分钟复跑 **15/15 PASS**。

---

6. 复现方式

```
:: 离线（不需要板子）
python host\encoder.py    --selftest        :: 10 项
python host\abz.py        --selftest        :: 15 项
python host\export_png.py --selftest        :: 11 项
python host\recorder.py   --selftest        :: 13 项
python host\signal.py     --selftest        :: 22 项
python host\regs.py       --selftest        :: 2043 组往返 + 5 条边界
python host\selftest_ui.py                  :: 30 项（offscreen，不弹窗）

:: 真机（COM3 @ 2 Mbaud）
python host\selftest_link.py --port COM3 --seconds 5
python host\selftest_s2.py   --port COM3 --seconds 6
python host\selftest_s3.py   --port COM3 --seconds 5 --clear
python host\selftest_s4.py   --port COM3 --seconds 8
python host\selftest_s5.py   --port COM3 --seconds 12
python host\selftest_s5.py   --port COM3 --seconds 12 --reconnect
python host\selftest_ui.py   --real-port COM3 --seconds 6
python host\selftest_s5.py   --port COM3 --soak-minutes 30 --dir host\_s5_soak
```

---

7. 未做与仍待定（如实列出）

项	状态	说明
30 分钟长稳（`link.py` 重写后）	**已跑完，15/15 PASS**	第一次跑到 1320 s 时被我**误判磁盘满**而中断（沙箱里 `Get-PSDrive C` 报 free 0.0 GB，实际还有 29 GB；应看 `DriveInfo` / `shutil.disk_usage`）⇒ 顺手给 `selftest_s5.py` 加**开记录前的盘空间预检**（不够就中文提示 + `exit 1`），**S5 判据 8 项 → 9 项**；完整 30 分钟：**17,859,119 帧 / 2.77 GB / 18 卷 / 丢帧 0.76% / 内存不涨 / 丢批 0**，见 `docs/DEV_STATUS.md` §1.66 与 `host\_s5_soak_run2.txt`
`abz.py` Z 脉宽口径（`∝ div` vs `docs/01:596`）	**待真机 ILA 定案**	代码里两种口径的分歧已写明，没有单方面改文档
`ports.py` 的 FTDI 分支（rank=1）	**无法在本机复现**	只有单元级核对；真板是 CH340（rank=0）
`signal.py:40` `LATCH_CLEAR_VALUE`	决定**不收窄**（0x00FF）	理由见 §5.4
`signal.py` 的 `VAL_BITS` b4（lagfatl）	保留分支 + 注释	`rtl/reg_file.v:248-249` 把 b4 硬接 0（地址表 `rtl/reg_file.v:79-80` 明写「b1/b4/b6/b7 恒 0」），`0x0404/0x0405` 看不到该位；真正的 `lag_fatal` 只从遥测 `io_flags` bit1 看